

HW06 — AI-Assisted API Testing Report

Student: Luong Linh Khoi · **ID:** 23127396 · **Repo:** [linhkhoy1309/eshop-sut-hw06-api](#) · **SUT:** EShop backend, vendored unmodified under `sut/` · **Tool:** Postman + Newman 6.2.x

1. Scope and Method

Three endpoints were selected, one per requirement pool, to avoid duplicating groupmates' selections (see `PLAN.md` §0 for the full non-duplication rationale):

Pool	Endpoint	Spec	Why this one
A	<code>PUT /api/users/me</code>	FR-04	An exact numeric boundary rule (phone: starts with 0, 10–11 digits) and a verbatim role-immutability rule (SEC-06) — an unambiguous oracle, not an inference
B	<code>POST /api/apply-coupon</code>	FR-09	Five stated business conditions (C1–C5) with an explicit discount formula and an explicit $\geq$ threshold — the richest partition/boundary surface in the SUT
C	<code>PUT /api/admin/orders/:id/status</code>	FR-10, FR-18	The order state machine directly (5 states, final-state rules) plus vertical privilege escalation on an admin route (SEC-03)

These three form one attack chain, not three unrelated tests: API 1's SEC-06 defect (a client can set its own `role`) promotes a plain user to admin, and API 3's SEC-03 defect (no route checks the token's role at all) means that promotion isn't even required to reach full admin control. Both are reported together as `BUG-01 + BUG-02` (GitHub Issue [#3](#)).

Every API went through the same four-stage, AI-first pipeline, each stage a real committed artefact:

- Generate** — six narrow, spec-anchored prompts (parameter inventory → domain partitions → state transitions → security → schema validation → consolidation), never one generic "generate all test cases" prompt. See `docs/generator-design.md` for why this decomposition matters and what fails without it.
- Audit** — every generated case labelled `VALID` / `INVALID` / `INCOMPLETE` against two oracles (the spec text and the SUT's source code, cited by line number), with corrections recorded next to the original rather than silently replacing it.
- Extend** — at least 5 human-added cases per API the generator was structurally incapable of producing, each tagged with *why* (prompt scope, model tendency, a fact only visible in source, or reasoning that spans two requests/two endpoints).
- Execute** — a Postman collection per API (plus a dedicated data-driven Collection Runner sweep each), run via Newman, with every failure traced to a specific, named defect.

2. Test Case Summary

See `README.md`'s Test Summary Report table for the full per-API breakdown (181 generated, 31 human-added, 212 executed, 161 passed, 51 deliberately-failing/quarantined). The generation floor (Requirement §6: "target ≥ 35 per API") was cleared by all three: 40/38/49 cases at the generation stage alone (before data-driven sweeps or extension), detailed in each API's own `generated.md`.

3. Postman Features Used

Feature	Where
Collections + folders	One collection per API, foldered by generation stage (S2...S5, EX Human Extensions)
Environment	EShop Local (HW06) (postman/environments/local.postman_environment.json), 30+ variables, secrets typed as secret
Pre-request scripts	Collection-level x-Student-Id header injection + console log (every collection); per-item token/state setup
Test scripts + Chai assertions	One pm.test per assertion, named by case ID, so Newman's assertion count reflects real case coverage rather than one blanket check per request
pm.sendRequest chaining	Multi-step cases (e.g. API 1's mutate-then-verify-via-GET, API 3's dynamic state-machine walk in the data-driven sweep) without needing separate collection items for every intermediate call
Data-driven runs (Collection Runner / newman -d)	postman/data/phone-cases.csv (API 1, digit-count × leading-0 matrix), postman/data/coupon-cases.csv (API 2, >= threshold swept across all 3 real coupons), postman/data/order-status-cases.csv (API 3, the full 5×5 FR-10 transition matrix, dynamically walked per iteration)
Newman CLI + newman-reporter-htmlextra	scripts/run-newman.js — orchestrates all 6 collections, re-seeding the SUT between each (see CLAUDE.md for why re-seeding is load-bearing, not cosmetic)
Newman in CI	.github/workflows/api-tests.yml, with a quarantine-aware gate (postman/known-defects.json) so a documented defect's failing assertion doesn't have to be silently weakened to keep the build green — see docs/cicd-report.md

Not exercised: workspaces, mock servers, and monitors were planned (PLAN.md §6) but dropped from scope in favour of the data-driven sweeps and the CI quarantine mechanism, which were judged higher-value within the time budget — stated here rather than left implicit.

4. Bugs Found

16 confirmed findings, filed as GitHub Issues [#1–#16](#), full detail and reproduction steps in bug-report.md .
Headline findings:

- **BUG-01 + BUG-02 (Critical, composed):** a brand-new, never-escalated account reaches full admin control of every order in two requests — API 1's role-mass-assignment defect composed with API 3's missing role check on every /api/admin/* route.
- **BUG-03 (Critical):** API 2's percent-discount formula is inverted (total * (1 - discount_value) instead of total × discount_value / 100) — a 10% coupon multiplies the bill by ten.
- **BUG-04 (Critical):** passwords are stored in plaintext and returned verbatim by GET /api/users/me .
- **BUG-05 (High):** canceled → delivered is explicitly whitelisted in API 3 despite canceled being a stated final state — composed with the legal pending → canceled arrow, any order reaches "delivered" in two hops while skipping confirmed / shipping entirely, which matters because FR-13's dashboard revenue sums total_amount over delivered orders.
- Six further candidate defects were surfaced **during the audit stage itself**, not from the original hypothesis list — e.g. apply-coupon 's per-user quota check can be bypassed indefinitely because the endpoint never

records a use itself (that happens via a separate, easy-to-forget endpoint), and API 3's transition whitelist omits `shipping → canceled` even though FR-10 explicitly grants Admin that exception.

Tested-and-passed negative results are recorded too, not silently dropped: `email` immutability (API 1), SQL-injection resistance on all three endpoints, `/api/coupon-usage` 's correct token-based identity scoping (API 2), and the admin order-update's correct scoping to a single row (API 3) — see `bug-report.md` 's closing section.

5. CI/CD

Full pipeline configuration, the quarantine mechanism design, and real green (`C1`) / red (`C2`) / reverted (`C3`) GitHub Actions run evidence with screenshots: `docs/cicd-report.md` .

6. AI Test-Generator Design (§7)

`docs/generator-design.md` + `generator.py` (pseudocode). Formalizes the six-stage pipeline actually run three times over into an orchestration design, including the two invariant rules every stage prompt carried (assert `UNDETERMINED` rather than guess; never infer conventional API behaviour for a deliberately-defective SUT) and the human-boundary line between generation and audit/extension that must not be collapsed.

On the diagram: `evidence/generator-diagram.jpg` is hand-drawn by the student, not AI-generated, per Requirement §7/§11 — embedded at the top of `docs/generator-design.md` .

7. AI Audit and Critique

`ai-audit-report.md` (18 logged interactions, chronological, each with the verbatim prompt, the output, and the human review verdict) and `ai-critique.md` (the required 200–300 word critique, centred on a mistake that recurred three times across the session — see that file for the specific instance and the operational principle drawn from it).